

Análisis del Coprocesador ORB_extractor

1. Objetivos

OBJ1. Identificación de los parámetros de diseño del copro

OBJ2. Depuración del copro

PRODUCTO: VERONTE GNSS DENIED																											
1º versión - PRODUCTO MÍNIMO VIABLE																											
		2023																									
		OCTUBRE				NOVIEMBRE				DICIEMBRE				ENERO				FEBRERO				MARZO					
TAREA	RESPONSABLE	S40	S41	S42	S43	S44	S45	S46	S47	S48	S49	S50	S51	S52	S1	S2	S3	S4	S5	S6	S7	S8	S9	S10	S11	S12	S13
1	Prototipo pipeline SW OK	★	OK																								
2	Pruebas en vuelo con Jetson									★																	
3	1ª versión SW en Zynq Ultrascale+ (ZUS+)																										
4	SW en Zynq Ultrascale+ (ZUS+) OK. Petalinux								OK	★																	
5	Análisis aceleración SW en FPGA					OK																					
	Subir documento de análisis en issue																										
6	Modificación SW para aceleración en FPGA																										
	6.1 Versión 1. Diseño de arquitectura PS-PL									★																	
	Sincronización/Buffering. Testeo pipeline-FPGA																										
	Configurar drivers cámara en Petalinux																										
	6.2 Versión 2. SW acelerado 1																										
	Diseño de referencia Wang 2021																										
	Acelerar Resize/¿FAST?																										
	Testeo pipeline - FPGA																										
	6.3 Versión 3. SW acelerado 2																										
	Acelerar ¿FAST?/BRIEF																										
	Testeo pipeline - FPGA																										
7	SW acelerado en FPGA																										
	8 Análisis aceleración en Avionics																										★

2. Visión general

El coprocesador está compuesto de tres IPs,

- Resize:
- FAST
- RS_BRIEF

diagrama con la info de los stream de entrada y salida

3. Parámetros de configuración de la arquitectura

Para cada IP comentar describir los parámetros así como las alternativas de implementación,

- Resize:
- FAST
- RS_BRIEF

diagrama con la info de los stream de entrada y salida

4. Análisis del código

Para cada IP describir las principales funciones

A) Resize

```

void resize(hls::stream<ap_axiu<32, 1, 1, 1> > &cfgStream, hls::stream<ap_axiu<INPUT_STREAM_BIT, 1, 1, 1> >
&srcStream, hls::stream<ap_axiu<32, 1, 1, 1> > &cfgoutStream, hls::stream<ap_axiu<OUTPUT_STREAM_BIT, 1, 1, 1>
> &outStream)
{
    #pragma HLS INTERFACE ap_ctrl_none port = return
    #pragma HLS INTERFACE axis register both port = cfgStream
    #pragma HLS INTERFACE axis register both port = srcStream
    #pragma HLS INTERFACE axis register both port = cfgoutStream
    #pragma HLS INTERFACE axis register both port = outStream

    process_cfg(cfgStream, cfgoutStream);
    if (scale == 1)
        process_scale_1(srcStream, outStream);
    else
        process(srcStream, outStream);
}

void process(hls::stream<ap_axiu<INPUT_STREAM_BIT, 1, 1, 1> > &srcStream,
hls::stream<ap_axiu<OUTPUT_STREAM_BIT, 1, 1, 1> > &outStream)
{
    process_input(srcStream, pixelData);
    process_buf(pixelData, outData);
    process_select(outData, selectData);
    process_output(selectData, outStream);
}

```

B) FAST

```

void FAST(hls::stream <ap_axiu<32, 1, 1, 1> > &cfgStream, hls::stream <ap_axiu<INPUT_STREAM_BIT, 1, 1, 1> >
&srcStream,
        hls::stream <ap_axiu<32, 1, 1, 1> > &cfgoutStream, hls::stream <ap_axiu<OUTPUT_STREAM_BIT, 1, 1, 1> >
&outPixelStream,
        hls::stream <ap_axiu<OUTPUT_STREAM_BIT, 1, 1, 1> > &outFASTStream)
{
    #pragma HLS INTERFACE ap_ctrl_none port = return
    #pragma HLS INTERFACE axis register both port = cfgStream
    #pragma HLS INTERFACE axis register both port = srcStream
    #pragma HLS INTERFACE axis register both port = cfgoutStream
    #pragma HLS INTERFACE axis register both port = outPixelStream
    #pragma HLS INTERFACE axis register both port = outFASTStream

    process_cfg(cfgStream, cfgoutStream);
    process(srcStream, outPixelStream, outFASTStream);
}

void process(hls::stream <ap_axiu<INPUT_STREAM_BIT, 1, 1, 1> > &srcStream,
        hls::stream <ap_axiu<OUTPUT_STREAM_BIT, 1, 1, 1> > &outPixelStream,
        hls::stream <ap_axiu<OUTPUT_STREAM_BIT, 1, 1, 1> > &outFASTStream)
{
    #pragma HLS DATAFLOW
    hls::stream <ap_uint<INPUT_BIT> > pixelData;
    #pragma HLS STREAM variable = pixelData depth = 2
    hls::stream <ap_uint<PROCESS_BIT> > gaussData;
    #pragma HLS STREAM variable = gaussData depth = 2
    hls::stream <ap_uint<PROCESS_BIT> > FASTData;
    #pragma HLS STREAM variable = FASTData depth = 2
    hls::stream <ap_uint<(WIN_SZ - 1) * PIXEL_BIT> > initData;
    #pragma HLS STREAM variable = initData depth = 2
    hls::stream <ap_uint<INPUT_BIT> > srcData;
    #pragma HLS STREAM variable = srcData depth = 2

    process_input(srcStream, pixelData);

```

```

process_padding(pixelData, initData, srcData);
process_buf(initData, srcData, gausData, FASTData);
process_output(gausData, FASTData, outPixelStream, outFASTStream);
}

```

TODO: Modificar el IP FAST para que el threshold sea un parámetro de configuración. Analizar la mejor opción de hacerlo: (i) a través de Interface AXI-Lite (ii) a través del stream de configuración Interfaz AXIStream

Revisado el código se emplea en este tipo de expresiones:

- uchar_t a0 = THRESHOLD;
- posJudge[iii] = (diff[iii] > THRESHOLD) ? 1 : 0;

El convertirlo en parámetro supondría un mínimo aumento de la lógica para implementar los comparadores y para almacenar el valor

C) RS-BRIEF

5. Depuración del pipeline ORB

A) Simulación HLS

Utilizando VivadoHLS se realiza una simulación C de los IPs y se generan los datos de cada etapa (Rsz, FAST, BRIEF). Se utilizan dos tamaños de imagen 640x480 y 12

Creado proyecto kria con rsz + FAST para depuración de las dos primeras etapas. Se conectan los streams de salida de FAST a dos DMAs y se leen independientemente desde el sw (PYNQ) para comparar con los resultados de la simulación.

Se han obtenido correctamente los datos del stream de keypoints y el stream de imagen (img blur).

Víctor me informa de que había un error en el Sw que transformaba los datos de los keypoints (se truncaban los bits MSB), esto provocaba que aparecieran menos keypoints. Ahora ya está subsanado, por lo que interrumpo la depuración del pipeline con la Kria.

TODO: comparar los resultados de simulación HLS con los resultados generados por el hw en la placa Alynx